

```
# I declare that my work contains no examples of misconduct, such as plagiarism, or collusion.
# Any code taken from other sources is referenced within my code solution.
# Student ID: w1960946 # Date:21/11/2023
```

```
from graphics import GraphWin, Point, Rectangle, Text
```

```
def validate_integer(prompt):
```

```
    while True:
        try:
            response = int(input(prompt))
            if response % 20 == 0:
                return response
            else:
                print("Out of range")
        except ValueError:
            print("Integer required")
```

```
def create_bar(win, x, y, width, height, label, value):
```

```
    bar = Rectangle(Point(x, y), Point(x + width, y + height))
    bar.setFill("Red")
    bar.draw(win) #display rectangle in application window (container)
```

```
    # Display the label above the bar
```

```
    label_point = Point(x + width / 2, y + 15) # text is in the middle since x / 2 from the bottom left of rectangle is in the middle of the rectangle
```

```
    label_text = Text(label_point, label) # text is lower than the bar as it adds 10 from the bottom left corner (see paint for ref)
    label_text.setSize(9)
    label_text.draw(win)
```

```
    value_point = Point(x + width / 2, y + height - 10)
```

```
    value_text = Text(value_point, value)
```

```
    value_text.setSize(9)
```

```
    value_text.draw(win)
```

```
def display_histogram(win, histogram_data):
```

```
    bar_width = 50
    bar_spacing = 40
    base_line = 250
```

```
    x = bar_spacing
```

```
    win.delete("all") # Clear previous content
```

```
    categories = ["Progress", "Module\nTrailer", "Module\nRetriever", "Exclude"]
```

```
    for i, category in enumerate(categories):
```

```
        create_bar(win, x, base_line, bar_width, 20 * histogram_data[i], category, histogram_data[i])
```

```
        x += bar_width + bar_spacing
```

```
def main():
```

```
    credit_range = [0, 20, 40, 60, 80, 100, 120]
```

```
    credits_data = []
```

```
    credits_grouped = [[], [], [], []]
```

```
    while True:
```

```
        pass_credits = validate_integer("Please enter your credits at pass: ")
```

```
        defer_credits = validate_integer("Please enter your credits at defer: ")
```

```
        fail_credits = validate_integer("Please enter your credits at fail: ")
```

```
        total_credits = pass_credits + defer_credits + fail_credits
```

```
        if total_credits == 120:
```

```
            credits_data.append((pass_credits, defer_credits, fail_credits))
```

```
            print("Total correct")
```

```
            if total_credits in credit_range:
```

```
                if fail_credits >= 80:
```

```
                    print("Exclude")
```

```
                    credits_grouped[3].append((pass_credits, defer_credits, fail_credits))
```

```
                elif pass_credits == 120:
```

```
                    print("Progress")
```

```
                    credits_grouped[2].append((pass_credits, defer_credits, fail_credits))
```

```
                elif pass_credits == 100:
```

```
                    print("Progress (module trailer)")
```

```
                    credits_grouped[1].append((pass_credits, defer_credits, fail_credits))
```

```
            else:
```

```
                print("Do not Progress - module retriever")
```

```
                credits_grouped[0].append((pass_credits, defer_credits, fail_credits))
```

```
        while True:
```

```
            user_input = input("Do you want to continue? (y/q): ").lower()
```

```
            if user_input == 'y':
```

```
                exit_loop = False
```

```
                break
```

```
            elif user_input == 'q':
```

```
                # We want to exit the big loop
```

```
                exit_loop = True
```

```
                print("Exiting the system")
```

```
                break
```

```
        else:
```

```
            print("Invalid input. Please enter 'y' to continue or 'q' to quit.")
```

```

        if exit_loop is True:
            break
        else:
            print("Out of range")
    else:
        print("Total incorrect")

# Prepare outcome strings
progress_str = f"Progress: {credits_grouped[2]}"
trailer_str = f"Progress (module trailer): {credits_grouped[1]}"
retriever_str = f"Do not Progress - module retriever: {credits_grouped[0]}"
exclude_str = f"Exclude: {credits_grouped[3]}"

# Print outcome strings
if credits_grouped[2]:
    print(progress_str)
if credits_grouped[1]:
    print(trailer_str)
if credits_grouped[0]:
    print(retriever_str)
if credits_grouped[3]:
    print(exclude_str)

# Calculate the histogram data
progress_count = 0
trailer_count = 0
retriever_count = 0
exclude_count = 0

for pass_credits, defer_credits, fail_credits in credits_data:
    if fail_credits >= 80:
        exclude_count += 1
    elif pass_credits == 120:
        progress_count += 1
    elif pass_credits == 100:
        trailer_count += 1
    else:
        retriever_count += 1

with open("test.txt", "w") as f:
    if credits_grouped[2]:
        f.write(progress_str + "\n")
    if credits_grouped[1]:
        f.write(trailer_str + "\n")
    if credits_grouped[0]:
        f.write(retriever_str + "\n")
    if credits_grouped[3]:
        f.write(exclude_str + "\n")

# Create a GraphWin
win = GraphWin("Credit Bar Graph", 400, 300)
win.setBackground("White")

# Display the histogram
histogram_data = [progress_count, trailer_count, retriever_count, exclude_count]
display_histogram(win, histogram_data)

# Display total number of students
total_students = len(credits_data)
print(f"Total number of students: {total_students}") #counting number of students on incoorect/not allowed data input

win.getMouse()
win.close()
main()

```